

Project Report

On

Computer Networking: Security

(CSE3752)

Image encryption and decryption using AES



Submitted by:

Name 1: JASHWANT KALUNDIA

Regd. No.:2341011003

Name 2: ANSUMAN MISHRA

Regd. No.:2341013160

Name 3: ADARSH MOHANTY

Regd. No.:2341013251

Name 4: ANKIT PARICHHA

Regd. No.:2341016173

Name 5: AMRUT RUP SAHOO

Regd. No.:2341019086

Name 6: SERON SENAPATI

Regd. No.:2341019378

B. Tech. **CSE** 6th Semester (Section **23412J1**)

INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH
(FACULTY OF ENGINEERING)
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY),
BHUBANESWAR, ODISHA

Declaration

We, the undersigned students of B. Tech. of CSE Department hereby declare that we own the full responsibility for the information, results etc. provided in this PROJECT titled “Image Encryption and Decryption using AES” submitted to **Siksha ‘O’ Anusandhan (Deemed to be University), Bhubaneswar** for the partial fulfillment of the subject **Computer Networking: Security (CSE 3752)**. We have taken care in all respect to honor the intellectual property right and have acknowledged the contribution of others for using them in academic purpose and further declare that in case of any violation of intellectual property right or copyright we, as the candidate(s), will be fully responsible for the same.

Name1: Jashwant Kalundia

Regd. No.: 2341011003

Name2: Ansuman Mishra

Regd. No.: 2341013160

Name3: Adarsh Mohanty

Regd. No.: 2341013251

Name4: Ankit Parichha

Regd. No.: 2341016173

Name5: Amrut Rup Sahoo

Regd. No.: 2341019086

Name6: Seron Senapati

Regd No.: 2341019378

Date:

Place: Bhubaneswar

Abstract

In the modern era of digital communication, images are widely transmitted through online platforms, cloud systems, social media applications, military communication systems, and healthcare networks. As the use of digital images increases, the risk of unauthorized access, cyberattacks, and image tampering also grows significantly. Therefore, image security has become an essential requirement in modern cybersecurity.

This project focuses on the implementation of image encryption and decryption using the Advanced Encryption Standard (AES) algorithm. AES is one of the most secure and efficient symmetric-key cryptographic algorithms used worldwide for protecting sensitive data. The project converts an original image into encrypted binary data using AES encryption and restores the image back to its original form through decryption using the same secret key.

The implementation is performed using the Python programming language along with libraries such as NumPy and PIL for image processing and matrix operations. The project demonstrates the practical application of AES-based cryptography in protecting multimedia data and ensuring secure communication over insecure networks.

Contents

Serial No.	Chapter No.	Title of the Chapter	Page No.
1.	1	Introduction	1
2.	2	Problem Statement	2
3.	3	Methodology	3-4
4.	4	Implementation	5-16
5.	5	Results and interpretation	17
6.	6	Conclusion	18
7.		References	19

1. Introduction

In today's digital world, multimedia content such as images, videos, and audio files are frequently exchanged over communication networks. Images are commonly used in online banking systems, military communication, social media platforms, confidential business applications, and healthcare systems. Since digital images may contain sensitive information, protecting them from unauthorized access is extremely important.

Traditional methods of storing and transmitting images are not secure against modern cyber threats. Hackers can intercept images during transmission or manipulate image content, resulting in data breaches and privacy violations. To solve these issues, encryption techniques are used to convert readable image data into an unreadable format.

Advanced Encryption Standard (AES) is one of the most powerful and secure encryption algorithms widely used for protecting digital information. AES uses symmetric key cryptography, meaning the same secret key is used for both encryption and decryption. The algorithm provides high security, fast execution speed, and resistance against brute-force attacks. This project aims to develop a secure image encryption and decryption system using AES to ensure confidentiality and secure image transmission.

2. Problem Statement

Digital images transmitted through communication channels are vulnerable to various cyber threats such as hacking, unauthorized access, interception, data theft, and tampering. Conventional methods of image storage and sharing do not provide sufficient protection against modern attacks. As the use of digital communication increases, maintaining image privacy and confidentiality has become a major challenge.

This project addresses these challenges by implementing an image encryption and decryption system using the AES algorithm. The system converts original image data into encrypted binary format, making it unreadable to unauthorized users. Only users with the correct AES secret key can decrypt and restore the image back to its original form.

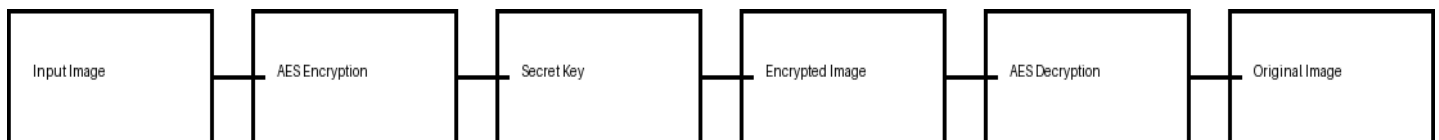
The project mainly focuses on ensuring:

- Confidentiality of image data
- Protection against unauthorized access
- Secure transmission of images over networks
- Resistance against cyberattacks and brute-force attacks

3. Methodology

I. Block Diagram of Cryptosystem Used

The following block diagram represents the working process of image encryption and decryption using AES. The input image is encrypted using AES encryption and a secret key to generate encrypted image data. The encrypted image is then decrypted using the same secret key to restore the original image.



II. Algorithm / Pseudocode for Implementation

The algorithm of the project is divided into two main processes: **image encryption** and **image decryption**. During encryption, the original image is converted into encrypted binary data using the Advanced Encryption Standard (AES) algorithm and a secure secret key. During decryption, the encrypted image is converted back into its original form using the same AES secret key. The system ensures confidentiality, integrity, and secure transmission of digital images over insecure communication channels.

Encryption Algorithm:

Step 1: Start

Step 2: Read the input image file from the system

Step 3: Input/Generate a secure 128-bit AES secret key

Step 4: Convert image data into binary byte array format

Step 5: Divide data into 16-byte blocks

Step 6: Apply padding to image data if required

Step 7: Encrypt image data using AES algorithm

Step 8: Combine all encrypted blocks

Step 9: Store the encrypted data in .enc format

Step 10: Save the generated AES key and original file size

Step 11: Stop

Decryption Algorithm:

Step 1: Start

Step 2: Read the encrypted .enc file from the system

Step 3: Read the original file size and the same 128-bit AES key

Step 4: Convert encrypted data into binary byte array format

Step 5: Divide the encrypted data into 16-byte blocks

Step 6: Apply inverse AES operations on each block

Step 7: Combine all decrypted blocks

Step 8: Remove extra padding bytes using the original file size

Step 9: Restore the original image file

Step 10: Save the decrypted image

Step 11: Stop

III. Tools Used for Implementation of Algorithm/Pseudocode

Various software tools and technologies are used in this project to implement the AES-based image encryption and decryption system efficiently. These tools assist in image processing, matrix manipulation, encryption, and decryption operations. Python programming language is used because of its simplicity, readability, and extensive support for scientific and image-processing libraries. The project manually implements the core AES transformations, including SubBytes, ShiftRows, MixColumns, and AddRoundKey operations, instead of using built-in AES encryption functions.

NumPy library is used for matrix operations and manipulation of image pixel data, while the Python Imaging Library (PIL) library is used for loading, processing, and saving image files. Secrets library is used to get a randomly generated 128 bits key. Development and testing of the project are carried out using Visual Studio Code and Jupyter Notebook, which provide an efficient programming and debugging environment.

Tools and Technologies Used

- Python Programming Language
- NumPy Library
- Python Image Library (PIL)
- Secrets Library
- AES-Based Image Encryption Algorithm
- Galois Field Multiplication
- Visual Studio Code / Jupyter Notebook

4. Implementation

The implementation of the project is carried out using the Python programming language along with NumPy and Python Imaging Library (PIL) libraries for image processing and matrix operations. The project manually implements the core AES transformations, including SubBytes, ShiftRows, MixColumns, and AddRoundKey operations, to perform AES-based image encryption and decryption.

Initially, the image file is loaded and converted into a byte array format. The image data is divided into 16-byte blocks, and a 128-bit secret key is used during the encryption process. Each block undergoes AES transformations to generate the encrypted image. During decryption, the same secret key is applied along with inverse AES operations to restore the original image.

The encrypted image appears visually unreadable, thereby ensuring confidentiality and security of the image data. The decrypted image is reconstructed and saved using a lossless image format to preserve image quality and integrity.

The following Python code demonstrates the implementation of AES-based image encryption and decryption.

```

# -----
# IMPORTING REQUIRED LIBRARIES
# -----

from PIL import Image
import numpy as np
import secrets

#-----
#ENCRYPTION

# -----
# AES S-BOX
# -----

S_BOX = [
    [0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76],
    [0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0],
    [0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15],
    [0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75],
    [0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84],
    [0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf],
    [0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8],
    [0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2],
    [0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73],
    [0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb],
    [0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79],
    [0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08],
    [0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a],
    [0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e],
    [0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf],
    [0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16]
]

# -----
# SUB BYTES

```

```
# -----
```

```
def sub_bytes(block):
```

```
    new_block = []
```

```
    for byte in block:
```

```
        row = byte >> 4
```

```
        col = byte & 0x0F
```

```
        new_block.append(S_BOX[row][col])
```

```
    return new_block
```

```
# -----
```

```
# SHIFT ROWS
```

```
# -----
```

```
def shift_rows(block):
```

```
    matrix = np.array(block).reshape(4,4)
```

```
    for i in range(4):
```

```
        matrix[i] = np.roll(matrix[i], -i)
```

```
    return matrix.flatten().tolist()
```

```
# -----
```

```
# GALOIS FIELD MULTIPLICATION
```

```
# -----
```

```
def gmul(a, b):
```

```
p = 0
```

```
for i in range(8):
```

```
    if b & 1:
```

```
        p ^= a
```

```
    high_bit = a & 0x80
```

```
    a <<= 1
```

```
    if high_bit:
```

```
        a ^= 0x1b
```

```
    a &= 0xFF
```

```
    b >>= 1
```

```
return p
```

```
# -----
```

```
# MIX COLUMNS
```

```
# -----
```

```
def mix_columns(block):
```

```
    matrix = np.array(block).reshape(4,4)
```

```
    for c in range(4):
```

```
        a0 = matrix[0][c]
```

```
        a1 = matrix[1][c]
```

```

a2 = matrix[2][c]
a3 = matrix[3][c]

matrix[0][c] = (gmul(a0,2) ^gmul(a1,3) ^gmul(a2,1) ^gmul(a3,1))
matrix[1][c] = (gmul(a0,1) ^gmul(a1,2) ^gmul(a2,3) ^gmul(a3,1))
matrix[2][c] = (gmul(a0,1) ^gmul(a1,1) ^gmul(a2,2) ^gmul(a3,3))
matrix[3][c] = (gmul(a0,3) ^gmul(a1,1) ^gmul(a2,1) ^gmul(a3,2))

return matrix.flatten().tolist()

# -----
# ADD ROUND KEY
# -----

def add_round_key(block, key):

    return [b ^ k for b, k in zip(block, key)]

# -----
# AES ROUND
# -----

def aes_round(block, key):

    block = sub_bytes(block)

    block = shift_rows(block)

    block = mix_columns(block)

    block = add_round_key(block, key)

```

```

    return block

# -----
# GENERATE RANDOM 128-BIT AES KEY
# -----

key = [secrets.randbelow(256) for _ in range(16)]

print("Generated AES 128-bit Key:")

print("".join(format(x, '02x') for x in key))

# -----
# LOAD IMAGE AS RAW BYTES
# -----

with open("input_image.png", "rb") as file:

    file_data = list(file.read())

original_size = len(file_data)

# -----
# IMAGE ENCRYPTION
# -----

encrypted_data = []

for i in range(0, len(file_data), 16):

    block = file_data[i:i+16]

```

```

while len(block) < 16:
    block.append(0)

encrypted_block = aes_round(block, key)

encrypted_data.extend(encrypted_block)

# -----
# SAVE ENCRYPTED FILE
# -----

with open("encrypted_image.enc", "wb") as file:

    file.write(bytes(encrypted_data))

print("\nAES Image Encryption Completed!")

#-----
#DECRYPTION

# -----
# GENERATE INVERSE S-BOX
# -----

INV_S_BOX = [[0]*16 for _ in range(16)]

for r in range(16):
    for c in range(16):

        value = S_BOX[r][c]

```

```

    INV_S_BOX[value >> 4][value & 0x0F] = (r << 4) | c

# -----
# INVERSE SUB BYTES
# -----

def inverse_sub_bytes(block):

    new_block = []

    for byte in block:

        row = byte >> 4
        col = byte & 0x0F

        new_block.append(INV_S_BOX[row][col])

    return new_block

# -----
# INVERSE SHIFT ROWS
# -----

def inverse_shift_rows(block):

    matrix = np.array(block).reshape(4,4)

    for i in range(4):
        matrix[i] = np.roll(matrix[i], i)

    return matrix.flatten().tolist()

# -----

```



```
# GALOIS FIELD MULTIPLICATION
```

```
# -----
```

```
def gmul(a, b):
```

```
    p = 0
```

```
    for i in range(8):
```

```
        if b & 1:
```

```
            p ^= a
```

```
        high_bit = a & 0x80
```

```
        a <<= 1
```

```
        if high_bit:
```

```
            a ^= 0x1b
```

```
        a &= 0xFF
```

```
        b >>= 1
```

```
    return p
```

```
# -----
```

```
# INVERSE MIX COLUMNS
```

```
# -----
```

```
def inverse_mix_columns(block):
```

```
    matrix = np.array(block).reshape(4,4)
```

```
for c in range(4):
```

```
    a0 = matrix[0][c]
```

```
    a1 = matrix[1][c]
```

```
    a2 = matrix[2][c]
```

```
    a3 = matrix[3][c]
```

```
    matrix[0][c] = (gmul(a0,14) ^ gmul(a1,11) ^ gmul(a2,13) ^ gmul(a3,9) )
```

```
    matrix[1][c] = (gmul(a0,9) ^ gmul(a1,14) ^ gmul(a2,11) ^ gmul(a3,13) )
```

```
    matrix[2][c] = (gmul(a0,13) ^ gmul(a1,9) ^ gmul(a2,14) ^ gmul(a3,11) )
```

```
    matrix[3][c] = (gmul(a0,11) ^ gmul(a1,13) ^ gmul(a2,9) ^ gmul(a3,14) )
```

```
return matrix.flatten().tolist()
```

```
# -----
```

```
# ADD ROUND KEY
```

```
# -----
```

```
def add_round_key(block, key):
```

```
    return [b ^ k for b, k in zip(block, key)]
```

```
# -----
```

```
# AES DECRYPTION ROUND
```

```
# -----
```

```
def decrypt_round(block, key):
```

```
    block = add_round_key(block, key)
```

```
    block = inverse_mix_columns(block)
```

```

    block = inverse_shift_rows(block)

    block = inverse_sub_bytes(block)

    return block

# -----
# ENTER SAME AES KEY USED DURING ENCRYPTION
# -----

key_hex = input("Enter AES Key: ")

key = [
    int(key_hex[i:i+2], 16)
    for i in range(0, len(key_hex), 2)
]

# -----
# LOAD ENCRYPTED FILE
# -----

with open("encrypted_image.enc", "rb") as file:

    encrypted_data = list(file.read())

    original_size = len(encrypted_data)

# -----
# IMAGE DECRYPTION
# -----

decrypted_data = []

```

```

for i in range(0, len(encrypted_data), 16):

    block = encrypted_data[i:i+16]

    while len(block) < 16:
        block.append(0)

    decrypted_block = decrypt_round(block, key)

    decrypted_data.extend(decrypted_block)

# -----
# REMOVE EXTRA PADDING
# -----

decrypted_data = decrypted_data[:original_size]

# -----
# SAVE DECRYPTED IMAGE
# -----

with open("decrypted_image.png", "wb") as file:

    file.write(bytes(decrypted_data))

print("\nAES Image Decryption Compl

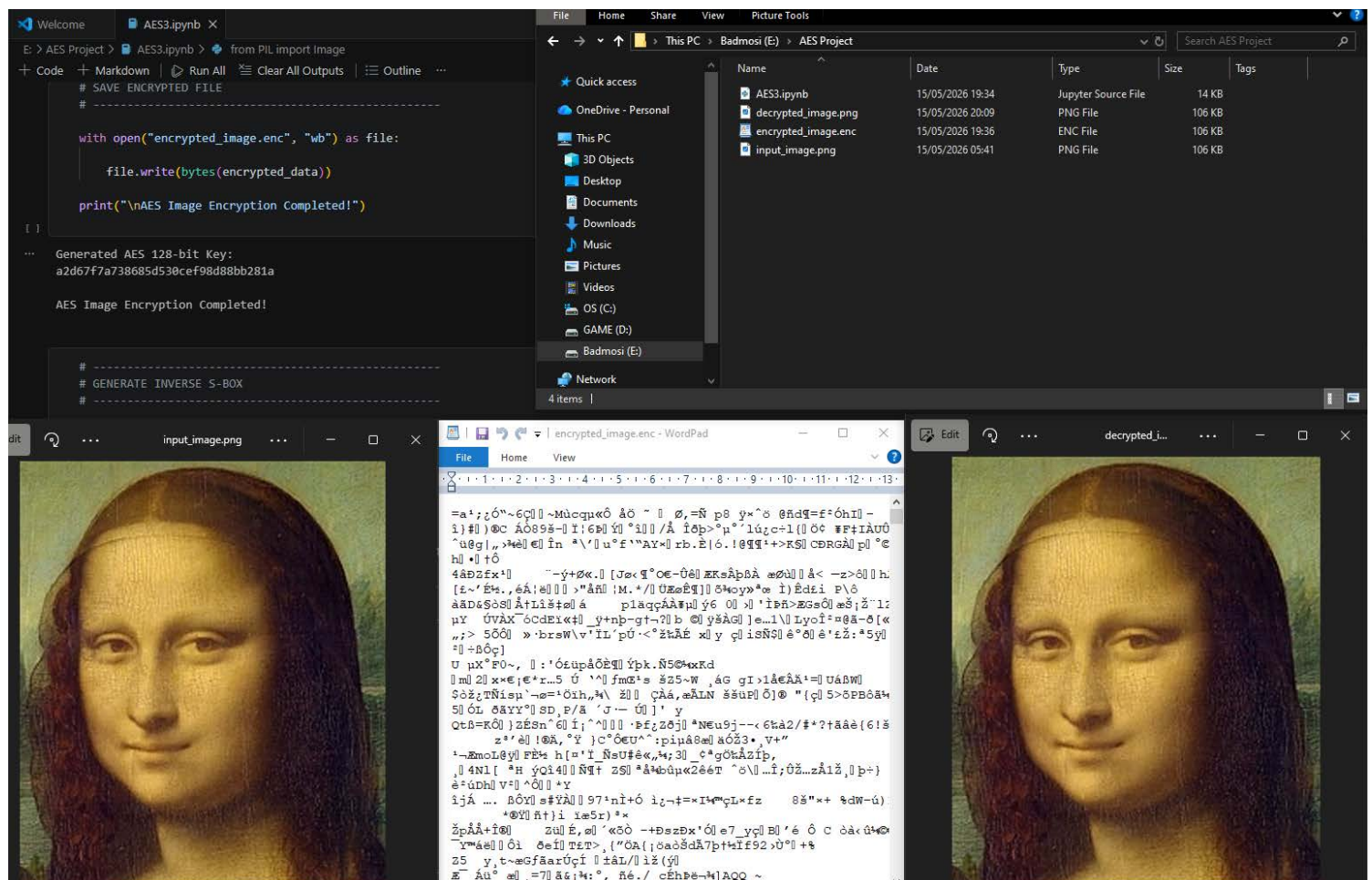
```

5. Results & Interpretation

The AES-based image encryption and Decryption system was successfully implemented and tested on sample image files. The original image was converted into encrypted binary data that appeared unreadable and meaningless to unauthorized users. The encrypted image could not be interpreted without the correct AES secret key, which demonstrates the effectiveness of AES in protecting image confidentiality. After decryption using the correct secret key, the original image was restored successfully without any data loss.

The results clearly show that AES provides:

- Strong security against cyberattacks
- Fast encryption and decryption speed
- Reliable performance for multimedia data protection
- Secure image communication over networks



6. Conclusion

The project “Image Encryption and Decryption Using AES” was successfully designed and implemented using the Python programming language along with NumPy, Python Imaging Library (PIL) and Secrets libraries. The developed system efficiently performs secure image encryption and decryption using AES-based cryptographic transformations, including SubBytes, ShiftRows, MixColumns and AddRound Key operations.

The original image is converted into byte data and encrypted using a 128-bit secret key, making the image visually unreadable during the encryption process. During decryption, inverse AES operations along with the same secret key are used to accurately restore the original image. The project demonstrates the practical application of AES concepts for securing digital image data and ensuring confidentiality during storage and transmission.

The project demonstrates the practical implementation of symmetric key cryptography for protecting digital images against unauthorized access, interception, and cyber threats. AES encryption provides strong security, fast processing speed, high reliability, and resistance against brute-force attacks, making it one of the most widely used encryption standards in modern cybersecurity systems.

Through this project, valuable knowledge was gained regarding cryptographic techniques, secure image processing, key management, and secure data transmission. The successful execution and testing of the system prove that AES is highly effective for maintaining confidentiality, integrity, and security of multimedia data over insecure communication networks.

Overall, the project highlights the importance of encryption technologies in modern digital communication and emphasizes the critical role of cybersecurity in protecting sensitive information in real-world applications such as banking systems, healthcare, military communication, cloud storage, and secure online data sharing.

References

(as per the IEEE recommendations)

[1] CompTIA Security+ SY0-701 Certification Guide by Ian Neil, 3rd Edition, Packt publication.

[2] Python Official Documentation.